

✓ Natural Language to SQL

```
!pip uninstall -y langchain
!pip install -q langchain langchain-community langchain-groq pandas sqlalchemy
```

```
Found existing installation: langchain 1.2.0
Uninstalling langchain-1.2.0:
  Successfully uninstalled langchain-1.2.0
```

```
import os
```

```
os.environ["GROQ_API_KEY"] = ""
```



✓ Create SQLite Database

```
import sqlite3

conn = sqlite3.connect("company.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS customers (
    customerNumber INTEGER PRIMARY KEY,
    customerName TEXT,
    country TEXT,
    creditLimit REAL
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS orders (
    orderNumber INTEGER PRIMARY KEY,
    customerNumber INTEGER,
    FOREIGN KEY (customerNumber) REFERENCES customers(customerNumber)
)
""")

cursor.executemany(
    "INSERT OR IGNORE INTO customers VALUES (?, ?, ?, ?)",
    [
        (103, "Atelier graphique", "France", 21000),
        (112, "Signal Gift Stores", "USA", 72000),
        (114, "Australian Collectors", "Australia", 117300),
        (119, "La Rochelle Gifts", "France", 45000)
    ]
)

cursor.executemany(
    "INSERT OR IGNORE INTO orders VALUES (?, ?)",
    [
        (10100, 103),
        (10101, 103),
        (10102, 112),
        (10103, 112),
        (10104, 112),
        (10105, 114)
    ]
)
```

```
conn.commit()
conn.close()

print("✅ Database created successfully")
```

✅ Database created successfully

✓ Create Table Metadata (Schema Context)

```
import pandas as pd

df = pd.DataFrame({
    "Table": ["customers", "orders"],
    "Description": [
        "Stores customer information including country and credit limit",
        "Stores orders placed by customers"
    ]
})

df.to_csv("database_table_descriptions.csv", index=False)
print("✅ Schema metadata CSV created")
```

✅ Schema metadata CSV created

✓ Load Schema Context

```
table_desc = pd.read_csv("database_table_descriptions.csv")

schema_info = ""
for _, row in table_desc.iterrows():
    schema_info += f"Table: {row['Table']}\nDescription: {row['Description']}\n\n"

print(schema_info)
```

```
Table: customers
Description: Stores customer information including country and credit limit

Table: orders
Description: Stores orders placed by customers
```

✓ Initialize NL2SQL Engine

```
from sqlalchemy import create_engine
from langchain_groq import ChatGroq
from langchain_community.utilities import SQLDatabase
from langchain_community.agent_toolkits import SQLDatabaseToolkit
from langchain_community.agent_toolkits.sql.base import create_sql_agent

# Connect to database
engine = create_engine("sqlite:///company.db")
db = SQLDatabase(engine)

# Groq LLM
llm = ChatGroq(
    model="llama3-70b-8192",
```

```

        temperature=0
    )

    # SQL Toolkit
    toolkit = SQLDatabaseToolkit(db=db, llm=llm)

    # SQL Agent
    agent = create_sql_agent(
        llm=llm,
        toolkit=toolkit,
        verbose=True,
        prefix=f"""
You are an expert NL2SQL assistant.
Use the following database schema to generate correct SQL queries.

{schema_info}
"""
    )

    print("🚀 NL2SQL system using Groq is ready!")

```

🚀 NL2SQL system using Groq is ready!

```

from langchain_groq import ChatGroq

llm = ChatGroq(
    model="llama-3.1-8b-instant",
    temperature=0
)

```

```

toolkit = SQLDatabaseToolkit(db=db, llm=llm)

agent = create_sql_agent(
    llm=llm,
    toolkit=toolkit,
    verbose=True,
    prefix=f"""
You are an expert NL2SQL assistant.
Use the following database schema to generate correct SQL queries.

{schema_info}
"""
)

```

```

from langchain_community.agent_toolkits import SQLDatabaseToolkit
from langchain_community.agent_toolkits.sql.base import create_sql_agent

toolkit = SQLDatabaseToolkit(db=db, llm=llm)

agent = create_sql_agent(
    llm=llm,
    toolkit=toolkit,
    verbose=True,
    prefix=f"""
You are an expert NL2SQL assistant.
Use the following database schema to generate correct SQL queries.

{schema_info}
"""
)

```

✓ Ask Your First Natural Language Query

```
question = "List customers and the number of orders they placed"
response = agent.invoke(question)
```

```
print("\nAnswer:\n", response["output"])
```

> Entering new SQL Agent Executor chain...

Thought: I need to list the tables in the database to see what I can query.

Action: sql_db_list_tables

Action Input: customers, orders Question: List customers and the number of orders they placed

Thought: I should look at the tables in the database to see what I can query. Then I should query

Action: sql_db_list_tables

Action Input: customers, orders Question: List customers and the number of orders they placed

Thought: I should query the schema of the relevant tables to see what fields are available.

Action: sql_db_schema

Action Input: customers, orders

```
CREATE TABLE customers (
    "customerNumber" INTEGER,
    "customerName" TEXT,
    country TEXT,
    "creditLimit" REAL,
    PRIMARY KEY ("customerNumber")
)
```

/*

3 rows from customers table:

customerNumber	customerName	country	creditLimit
103	Atelier graphique	France	21000.0
112	Signal Gift Stores	USA	72000.0
114	Australian Collectors	Australia	117300.0

*/

```
CREATE TABLE orders (
    "orderNumber" INTEGER,
    "customerNumber" INTEGER,
    PRIMARY KEY ("orderNumber"),
    FOREIGN KEY("customerNumber") REFERENCES customers ("customerNumber")
)
```

/*

3 rows from orders table:

orderNumber	customerNumber
10100	103
10101	103
10102	112

**/Question: List customers and the number of orders they placed*

Thought: I need to write a SQL query that joins the customers and orders tables to get the desired

Action: sql_db_query_checker

Action Input: SELECT customerName, COUNT(orderNumber) FROM customers JOIN orders ON customers.cust

Thought: I need to write a SQL query that joins the customers and orders tables to get the desired

Action: sql_db_query

Action Input: SELECT customerName, COUNT(orderNumber) FROM customers JOIN orders ON customers.cust

> Finished chain.

Answer:

```
[('Atelier graphique', 2), ('Australian Collectors', 1), ('Signal Gift Stores', 3)]
```

✓ Rephrasing SQL Results (Enhanced Clarity)

```
from langchain_core.prompts import ChatPromptTemplate
```

```
# Assuming 'results' still holds the last SQL query output
sql_output_to_rephrase = str(results)
```

```
rephrase_template = ChatPromptTemplate.from_messages([
```

```

rephrase_template = ChatPromptTemplate.from_messages([
    ("human", "Rephrase the following SQL output into a clear, user-friendly response, making sure
    ])

# Create a simple runnable for the LLM and the prompt
rephrase_runnable = rephrase_template | llm

# Invoke the runnable to get the rephrased answer
final_rephrased_answer = rephrase_runnable.invoke({"sql_result": sql_output_to_rephrase})

print("\nRephrased Answer (using direct LLM invocation):\n", final_rephrased_answer.content)

```

Rephrased Answer (using direct LLM invocation):
Based on our analysis, the following customers have placed orders:

- Atelier graphique: 2 orders
- Australian Collectors: 1 order
- Signal Gift Stores: 3 orders

This information provides a clear breakdown of the number of orders placed by each customer.

✓ Few-Shot Learning

```

few_shot_examples = """
Q: How many customers are there?
SQL: SELECT COUNT(*) FROM customers;

Q: Show customers from France
SQL: SELECT * FROM customers WHERE country = 'France';
"""

agent_fs = create_sql_agent(
    llm=llm,
    toolkit=toolkit,
    verbose=True,
    prefix=f"""
You are an expert SQL assistant.
Here are some example queries:

{few_shot_examples}

Use them as guidance.
Schema:
{schema_info}
"""
)

agent_fs.invoke("Which customers are from USA?")

```

```

> Entering new SQL Agent Executor chain...
Thought: I need to know the list of tables in the database to decide which ones to query.
Action: sql_db_list_tables
Action Input: customers, orders
Action: sql_db_schema
Action Input: customers, orders
CREATE TABLE customers (
    "customerNumber" INTEGER,
    "customerName" TEXT,
    country TEXT,
    "creditLimit" REAL,
    PRIMARY KEY ("customerNumber")
)

/*

```

```

3 rows from customers table:
customerNumber  customerName      country creditLimit
103      Atelier graphique      France  21000.0
112      Signal Gift Stores      USA      72000.0
114      Australian Collectors    Australia 117300.0
*/

CREATE TABLE orders (
    "orderNumber" INTEGER,
    "customerNumber" INTEGER,
    PRIMARY KEY ("orderNumber"),
    FOREIGN KEY("customerNumber") REFERENCES customers ("customerNumber")
)

/*
3 rows from orders table:
orderNumber      customerNumber
10100      103
10101      103
10102      112
*/Thought: Now that I have the schema of the customers and orders tables, I can see that the custo
Action: sql_db_query_checker
Action Input: SELECT * FROM customers WHERE country = 'USA';SELECT * FROM customers WHERE country
Thought: I now know the final answer
Action:
Final Answer: SELECT * FROM customers WHERE country = 'USA';

> Finished chain.
{'input': 'Which customers are from USA?',
 'output': "SELECT * FROM customers WHERE country = 'USA';"}

```

✓ Memory for Follow-Up Questions

```

question = "Show customers who placed no orders"
response = agent.invoke(question)

print("\nAnswer:\n", response["output"])

```

```

> Entering new SQL Agent Executor chain...
Thought: I need to list the tables in the database to see what I can query.
Action: sql_db_list_tables
Action Input: customers, ordersThought: Now that I know the tables in the database, I should query
Action: sql_db_schema
Action Input: customers, orders
CREATE TABLE customers (
    "customerNumber" INTEGER,
    "customerName" TEXT,
    country TEXT,
    "creditLimit" REAL,
    PRIMARY KEY ("customerNumber")
)

/*
3 rows from customers table:
customerNumber  customerName      country creditLimit
103      Atelier graphique      France  21000.0
112      Signal Gift Stores      USA      72000.0
114      Australian Collectors    Australia 117300.0
*/

CREATE TABLE orders (
    "orderNumber" INTEGER,
    "customerNumber" INTEGER,
    PRIMARY KEY ("orderNumber"),
    FOREIGN KEY("customerNumber") REFERENCES customers ("customerNumber")
)

```

```

)

/*
3 rows from orders table:
orderNumber    customerNumber
10100    103
10101    103
10102    112
*/Thought: Now that I know the schema of the tables, I can write a query to show customers who place orders.

Action: sql_db_query_checker
Action Input: SELECT * FROM customers LEFT JOIN orders ON customers.customerNumber = orders.customerNumber
Thought: I should check if the query is correct before executing it.
Action: sql_db_query_checker
Action Input: SELECT * FROM customers LEFT JOIN orders ON customers.customerNumber = orders.customerNumber
Thought: I should check if the query is correct before executing it.
Action: sql_db_query_checker
Action Input: SELECT * FROM customers LEFT JOIN orders ON customers.customerNumber = orders.customerNumber
Thought: I should check if the query is correct before executing it.

Final Answer: SELECT * FROM customers LEFT JOIN orders ON customers.customerNumber = orders.customerNumber

> Finished chain.

Answer:
SELECT * FROM customers LEFT JOIN orders ON customers.customerNumber = orders.customerNumber WHERE

```

```

sql_query = "SELECT customerName, COUNT(orderNumber) FROM customers JOIN orders ON customers.customerNumber = orders.customerNumber"
results = db.run(sql_query)
print("\nSQL Query Results:\n", results)

```

SQL Query Results:

```

[('Atelier graphique', 2), ('Australian Collectors', 1), ('Signal Gift Stores', 3)]

```

```

question = "List customers and the number of orders they placed"
response = agent.invoke(question)

print("\nAnswer:\n", response["output"])

```

```

> Entering new SQL Agent Executor chain...
Thought: I need to list the tables in the database to see what I can query.
Action: sql_db_list_tables
Action Input: customers, ordersQuestion: List customers and the number of orders they placed
Thought: I should look at the tables in the database to see what I can query. Then I should query the tables.
Action: sql_db_list_tables
Action Input: customers, ordersQuestion: List customers and the number of orders they placed
Thought: I should query the schema of the relevant tables to see what fields are available.
Action: sql_db_schema
Action Input: customers, orders
CREATE TABLE customers (
    "customerNumber" INTEGER,
    "customerName" TEXT,
    country TEXT,
    "creditLimit" REAL,
    PRIMARY KEY ("customerNumber")
)

/*
3 rows from customers table:
customerNumber    customerName    country    creditLimit
103    Atelier graphique    France    21000.0
112    Signal Gift Stores    USA    72000.0
114    Australian Collectors    Australia    117300.0
*/

CREATE TABLE orders (
    "orderNumber" INTEGER,
    "customerNumber" INTEGER,

```

```

PRIMARY KEY ("orderNumber"),
FOREIGN KEY("customerNumber") REFERENCES customers ("customerNumber")
)

/*
3 rows from orders table:
orderNumber    customerNumber
10100         103
10101         103
10102         112
*/
Question: List customers and the number of orders they placed
Thought: I need to write a SQL query that joins the customers and orders tables based on the cus
Thought: I should check the query to make sure it's correct before executing it.
Action: sql_db_query_checker
Action Input: SELECT customerName, COUNT(orderNumber) FROM customers JOIN orders ON customers.cu
Thought: I should check the query to make sure it's correct before executing it.
Action: sql_db_query_checker
Action Input: SELECT customerName, COUNT(orderNumber) FROM customers JOIN orders ON customers.cu
Thought: I now know the final answer
Final Answer: SELECT customerName, COUNT(orderNumber) FROM customers JOIN orders ON customers.cu

> Finished chain.

Answer:
SELECT customerName, COUNT(orderNumber) FROM customers JOIN orders ON customers.customerNumber

```

Sample Questions You Can Ask

1. Which customer has the highest credit limit?
2. Show customers who placed no orders
3. How many orders were placed in total?
4. Which country has the most customers?